LAB Manual – 2

Code/Course: 23CSE312/Distributed Systems

Topic: Shared Memory Computation Model

[CO05] [BTL-3]
Student Name: D Ganeswara
Roll No.: AM.SC.U4CSE23119

Batch/Section: CSE-B
Date: 11-02-2026

1 Task 1

Simulate shared memory communication models in a distributed environment. Implement a simple shared memory model in Python using multiprocessing.
Steps:

1. Create a shared memory object (using Value or Array).
2. Spawn multiple processes using multiprocessing.
3. Modify and access shared data between processes.
4. Synchronize access to the shared data, if necessary, using locks.

The multiprocessing module in Python allows you to run multiple processes in parallel on the same machine. The processes can share data in shared memory objects (like Value or Array) using locks to ensure safe access to shared data.
Shared memory is a concept that allows processes to access common data that resides in the same physical machine (laptop in this case). Even though they run concurrently, access to shared data can be controlled to prevent conflicts.
The multiprocessing module takes care of managing multiple processes on your laptop, creating separate memory spaces for each process but allowing certain variables (like the shared memory Value object) to be accessible across processes.

Expected Output

Process Incrementer 1 starting...
Process Incrementer 1 incremented value to 1 Process
Incrementer 2 starting...
Process Incrementer 2 incremented value to 2 Process Printer
reading value: 2
Final shared value: 2

Interpretation

Process 1 and Process 2 increment the shared value, and Process 3 prints it. Since the lock is used, the shared value is modified safely by one process at a time.

Results

```
from multiprocessing import Process, Value , Lock
import time


def incrementer(shared_value, lock , name):
   print(f"Process {name} is starting....")

   with lock:
       temp = shared_value.value
       time.sleep(1)
       shared_value.value = temp + 1
       print(f"Process {name} incremented value")

def printer(shared_value):
   time.sleep(2)
   print(f"Process printer reading value: : {shared_value.value}")

if __name__ == "__main__":
   shared_value = Value('i', 0)
   lock = Lock()

   p1 = Process(target=incrementer, args=(shared_value, lock, 'Incrementer 1'))
   p2 = Process(target=incrementer, args=(shared_value, lock, 'Incrementer 2'))
   p3 = Process(target=printer, args=(shared_value,))

   p1.start()
   p2.start()
   p3.start()

   p1.join()
   p2.join()
   p3.join()
```

```
    print("Final shared value:", shared_value.value)
```

```
❯ python3 task1.py
Process Incrementer 1 is starting....
Process Incrementer 2 is starting....
Process Incrementer 1 incremented value
Process printer reading value: : 1
Process Incrementer 2 incremented value
Final shared value: 2
```

2 Task 2: Extension — Shared Memory with Multiple Data Types

Use different shared memory objects (e.g., integers, floats, and strings) and modify them using different processes.

Steps:

1. Create a shared memory array that holds multiple data types (e.g., integers, floats, and strings).
2. Each process will modify one data type (e.g., Process 1 modifies the integer, Process 2 modifies the float, etc.).
3. Synchronize access using locks to ensure that no data corruption occurs.

Expected Output

Process 1 modified integer to 1
Process 2 modified float to 3.14 Process 3
modified string to 'Hello' Final array: [1, 3.14,
'Hello']

Results

```
from multiprocessing import Process, Value, Array, Lock

def modify_integer(shared_int, lock):
    with lock:
        shared_int.value = 1
        print("Process 1 modified integer to", shared_int.value)

def modify_float(shared_float, lock):
    with lock:
        shared_float.value = 3.14
        print("Process 2 modified float to", shared_float.value)

def modify_string(shared_string, lock):
    with lock:
        message = "Hello"
        shared_string.value = message.encode()
```

```python
        print("Process 3 modified string to", message)

if __name__ == "__main__":

    # Create shared memory objects
    shared_int = Value('i', 0)
    shared_float = Value('d', 0.0)
    shared_string = Array('c', 20)

    lock = Lock()
    p1 = Process(target=modify_integer, args=(shared_int, lock))
    p2 = Process(target=modify_float, args=(shared_float, lock))
    p3 = Process(target=modify_string, args=(shared_string, lock))
    p1.start()
    p2.start()
    p3.start()

    # Wait for completion
    p1.join()
    p2.join()
    p3.join()
    final_string = shared_string.value.decode().strip('\x00')

    print("Final array:", [shared_int.value, shared_float.value, final_string])
```

```
❯ python3 task2.py
Process 3 modified string to Hello
Process 2 modified float to 3.14
Process 1 modified integer to 1
Final array: [1, 3.14, 'Hello']
```